

S.A.G.E. - Sistema di Analisi e Gestione Economica

Corso: Basi di Dati
Anno accademico 2025/2026

Componenti del gruppo

Antonio Scharmuller
Riccardo Dallacasa

Matricole

Matricola 000000
Matricola 000000

Email universitarie

antonio.scharmuller@studio.unibo.it
riccardo.dallacasa@studio.unibo.it

Docente: Nome Cognome

11 maggio 2026

Indice

1	Introduzione	1
1.1	Obiettivo del progetto	1
1.2	Descrizione generale del sistema	1
1.3	Ambito del sistema	1
1.4	Utenti e ruoli previsti	1
1.5	Tecnologie utilizzate	1
1.6	Scelte progettuali preliminari	1
2	Analisi dei requisiti	2
2.1	Requisiti in linguaggio naturale	2
2.2	Analisi delle ambiguità e decisioni progettuali	2
2.3	Specifiche ristrutturate	2
2.4	Glossario dei termini essenziali	2
2.5	Classi di utenza / viste del sistema	3
2.6	Elenco delle operazioni principali	3
2.7	Vincoli espliciti e impliciti	3
3	Progettazione concettuale	3
3.1	Criteri generali di modellazione	3
3.2	Raffinamenti dello schema	3
3.2.1	Gerarchia delle transazioni (ISA)	3
3.2.2	Modellazione ibrida (Sistema vs Personale)	3
3.2.3	Reificazione del budget	3
3.3	Criticità rilevate	4
3.4	Schema scheletro E/R	5
3.5	Sviluppo delle viste o degli ambiti	5
3.5.1	Vista Studente e portafoglio personale	5
3.5.2	Vista classificazione delle transazioni	5
3.5.3	Vista budget e pianificazione	5
3.5.4	Vista spese ricorrenti e documenti	5
3.5.5	Vista amministratore e cataloghi globali	5
3.6	Raffinamenti proposti	5
3.7	Gerarchie presenti nello schema	5
3.8	Associazioni principali e cardinalità	5
3.9	Vincoli non rappresentabili direttamente nello schema E/R	5
3.10	Integrazione delle viste	5
3.11	Schema E/R finale	5
3.12	Dizionario di entità e associazioni	5
4	Specifiche funzionali	5
4.1	Elenco delle funzionalità richieste	5
4.2	Operazioni di inserimento	5
4.3	Operazioni di modifica	5
4.4	Operazioni di eliminazione	5
4.5	Operazioni di interrogazione	5
4.6	Operazioni statistiche / amministrative	5
4.7	Input, output e tabelle coinvolte per ogni operazione	5

5	Progettazione logica	5
5.1	Stima del volume dei dati	5
5.2	Descrizione delle operazioni principali e frequenza	5
5.3	Schemi di navigazione	5
5.4	Tabelle degli accessi	5
5.5	Analisi delle ridondanze	5
5.6	Raffinamento dello schema	5
5.6.1	Eliminazione delle gerarchie	5
5.6.2	Soluzione logica anticipata	5
5.6.3	Eliminazione degli attributi composti	6
5.6.4	Gestione degli attributi multivalore	6
5.6.5	Scelta delle chiavi primarie	6
5.6.6	Traduzione delle associazioni	6
5.7	Traduzione di entità e associazioni in relazioni	6
5.8	Schema relazionale finale	6
5.9	Vincoli di integrità	6
5.10	Controllo della normalizzazione	6
6	Implementazione SQL	6
6.1	Script di creazione del database	6
6.2	Creazione delle tabelle	6
6.3	Definizione di chiavi primarie e chiavi esterne	6
6.4	Vincoli NOT NULL, UNIQUE, CHECK	6
6.5	Trigger, procedure o viste, se usati	6
6.6	Popolamento iniziale del database	6
6.7	Query SQL delle operazioni principali	6
6.7.1	Problemi e criticità	6
6.7.2	Analisi	7
6.7.3	Soluzione SQL	7
6.7.4	Query di supporto e operazioni amministrative	7
6.7.5	Motivazioni	10
7	Progettazione dell'applicazione	10
7.1	Architettura generale dell'applicazione	10
7.2	Tecnologie usate	10
7.3	Struttura dei package Java	10
7.4	Collegamento al database tramite JDBC	10
7.5	Organizzazione delle classi principali	10
7.6	Gestione degli errori e validazione input	10
7.7	Interfaccia grafica Java Swing	10
7.8	Schermate principali	10
7.9	Flussi d'uso principali	11
8	Test e validazione	11
8.1	Dati di test utilizzati	11
8.2	Test delle query SQL	11
8.3	Test delle funzionalità applicative	11
8.4	Casi limite gestiti	11
8.5	Errori noti o limitazioni	11

9 Conclusioni	11
9.1 Risultato ottenuto	11
9.2 Scelte progettuali principali	11
9.3 Possibili sviluppi futuri	11
A Script SQL completo	11
B Query SQL complete	11
C Screenshot dell'applicazione	11
D Manuale utente sintetico	11
E Eventuale link al repository	12

1 Introduzione

1.1 Obiettivo del progetto

La proposta iniziale di S.A.G.E. è la realizzazione di un sistema per la gestione e l'analisi delle finanze personali universitarie. L'obiettivo è supportare lo studente nella registrazione, organizzazione e analisi delle proprie transazioni economiche, offrendo strumenti per classificazione, pianificazione e monitoraggio nel tempo.

1.2 Descrizione generale del sistema

S.A.G.E. è progettato come un'applicazione multiutente in cui ogni studente dispone di un portafoglio finanziario personale isolato. Il sistema consente la registrazione di spese e entrate, la categorizzazione tramite categorie e tag, la definizione di budget temporali e la gestione di documenti digitali associati alle transazioni.

1.3 Ambito del sistema

L'ambito del progetto comprende:

- gestione individuale del portafoglio economico personale;
- classificazione delle transazioni tramite categorie gerarchiche e tag trasversali;
- gestione dei budget mensili e per categoria;
- supporto per spese ricorrenti come modelli astratti;
- associazione di documenti digitali alle spese;
- analisi e reportistica aggregata.

Non sono previste funzionalità di condivisione di spese o conti comuni tra utenti.

1.4 Utenti e ruoli previsti

Il sistema prevede due ruoli principali:

- **Utente:** gestisce le proprie transazioni, categorie e tag personali, definisce budget, registra documenti e visualizza report finanziari.
- **Amministratore:** gestisce gli elementi globali del sistema (categorie, tag, fonti) e analizza dati aggregati anonimi senza accedere ai dettagli finanziari personali degli utenti.

1.5 Tecnologie utilizzate

DBMS: MySQL

Linguaggio applicativo: Java

Interfaccia: Java Swing

Connessione DB: JDBC

1.6 Scelte progettuali preliminari

Fin dalla fase iniziale sono state adottate alcune scelte progettuali rilevanti: l'isolamento dei dati finanziari tra utenti, la distinzione tra elementi di classificazione globali e personali e la memorizzazione dei documenti digitali tramite percorso file anziché tramite BLOB nel database. Tali decisioni mirano a garantire privacy, personalizzazione e maggiore semplicità di gestione del DBMS.

2 Analisi dei requisiti

2.1 Requisiti in linguaggio naturale

Il sistema deve supportare la registrazione e il monitoraggio dei movimenti finanziari personali di uno studente universitario, distinguendo tra spese ed entrate. Ogni transazione deve avere importo, data e descrizione. Le spese devono essere associate a una categoria, mentre le entrate devono essere associate a una fonte. Il sistema deve permettere la definizione di budget mensili e per categoria, nonché la gestione di spese ricorrenti tramite un modello astratto. Deve inoltre conservare il percorso dei documenti associati alle spese senza memorizzare i file nel database.

2.2 Analisi delle ambiguità e decisioni progettuali

Termine/Concetto	Criticità/Ambiguità	Decisione progettuale
Periodo temporale	Come associare una spesa a un budget mensile?	Il periodo è derivato dalla data della transazione. L'entità PERIODO serve per la storicizzazione dei budget.
Spesa ricorrente	Generazione dei record nel DB?	È un modello astratto; la logica Java crea nuove SPESE a partire dai parametri del modello.
Privacy/Admin	L'admin vede i dati degli utenti?	Vincolo applicativo: l'admin accede solo a viste aggregate (COUNT/SUM), non ai dati granulari.
Documenti	Dove salvare i file?	Si salva solo il percorso sul file system; il DB non gestisce BLOB per evitare overhead.

2.3 Specifiche ristrutturare

- R1: Gestione delle transazioni finanziarie (spese/entrate) con dettagli minimi obbligatori.
- R2: Classificazione delle spese tramite categorie e classificazione trasversale delle transazioni tramite tag.
- R3: Definizione e storico dei budget mensili e dei budget per categoria.
- R4: Supporto per spese ricorrenti come modello di generazione automatica.
- R5: Salvare solo il percorso dei documenti digitali associati alle spese, senza memorizzare i file nel database.
- R6: Accesso amministrativo limitato a statistiche aggregate, senza visibilità sui dati finanziari personali.

2.4 Glossario dei termini essenziali

- **Transazione:** entità astratta che rappresenta un movimento monetario con ID, data, importo e descrizione.
- **Budget:** reificazione della soglia di spesa legata a Utente, Categoria e Periodo.
- **Tag:** etichetta trasversale associabile a una o più transazioni, utile per analisi granulari non limitate alla sola categoria.
- **Categoria:** classificazione gerarchica 1:N applicata alle spese.
- **Fonte:** origine finanziaria obbligatoria per le entrate.

2.5 Classi di utenza / viste del sistema

- **Utente/Studente:** accede al proprio portafoglio, registra transazioni, gestisce categorie personali e visualizza report.
- **Amministratore:** gestisce categorie/tag di sistema e visualizza statistiche aggregate.

Le viste del sistema includono il portafoglio personale, il cruscotto dei budget e la dashboard degli andamenti finanziari.

2.6 Elenco delle operazioni principali

- Inserimento di una nuova spesa.
- Inserimento di una nuova entrata.
- Creazione/modifica/eliminazione di categorie personali.
- Creazione/modifica/eliminazione di tag personali.
- Definizione e aggiornamento dei budget mensili e per categoria.
- Gestione delle spese ricorrenti.
- Associazione di un documento digitale a una spesa.
- Visualizzazione di report aggregati e statistiche.

2.7 Vincoli espliciti e impliciti

- L'amministratore può consultare solo dati aggregati (COUNT/SUM) e non i dettagli finanziari degli utenti.
- I documenti vengono memorizzati solo tramite il percorso file, non come BLOB nel database.
- Ogni transazione deve contenere data, importo e descrizione.
- Il periodo del budget è derivato dal mese/anno della transazione e viene storicizzato tramite l'entità PERIODO.
- La spesa ricorrente è un modello astratto che genera transazioni effettive, non una transazione reale.

3 Progettazione concettuale

3.1 Criteri generali di modellazione

3.2 Raffinamenti dello schema

3.2.1 Gerarchia delle transazioni (ISA)

Abbiamo una specializzazione totale ed esclusiva con super-classe TRANSAZIONE e sotto-classi SPESA e ENTRATA. La sottoclasse SPESA è associata obbligatoriamente a una categoria, mentre la sottoclasse ENTRATA è associata obbligatoriamente a una fonte. I tag sono invece collegati alla super-entità TRANSAZIONE, in quanto rappresentano una classificazione trasversale utilizzabile sia per spese sia per entrate.

3.2.2 Modellazione ibrida (Sistema vs Personale)

Per CATEGORIA, TAG e FONTE si applica una gerarchia ISA: elementi di sistema creati dall'admin e elementi personali creati dall'utente.

3.2.3 Reificazione del budget

Il budget è un'entità che mette in relazione utente e periodo, con un eventuale riferimento a una categoria. L'assenza della categoria rappresenta un budget mensile complessivo, mentre la presenza della categoria rappresenta un budget specifico. Questa progettazione consente di

storicizzare i limiti di spesa e di fissare soglie per ambiti specifici. Nel capitolo di progettazione logica verrà valutata l'introduzione dell'attributo ridondante `Totale_Speso_Attuale` nel budget per migliorare le prestazioni di lettura nella dashboard.

3.3 Criticità rilevate

- La fonte deve essere legata esclusivamente a ENTRATA; una spesa ha una categoria di uscita.
- La spesa ricorrente punta a CATEGORIA, non a SPESA, perché è un modello generatore di tuple e non una transazione già avvenuta.
- L'utente viene identificato tramite un identificatore surrogato `ID_Utente`. L'email viene mantenuta come attributo con vincolo UNIQUE, ma non viene usata come chiave primaria.

- 3.4 Schema scheletro E/R
- 3.5 Sviluppo delle viste o degli ambiti
 - 3.5.1 Vista Studente e portafoglio personale
 - 3.5.2 Vista classificazione delle transazioni
 - 3.5.3 Vista budget e pianificazione
 - 3.5.4 Vista spese ricorrenti e documenti
 - 3.5.5 Vista amministratore e cataloghi globali
- 3.6 Raffinamenti proposti
- 3.7 Gerarchie presenti nello schema
- 3.8 Associazioni principali e cardinalità
- 3.9 Vincoli non rappresentabili direttamente nello schema E/R
- 3.10 Integrazione delle viste
- 3.11 Schema E/R finale
- 3.12 Dizionario di entità e associazioni

4 Specifiche funzionali

- 4.1 Elenco delle funzionalità richieste
- 4.2 Operazioni di inserimento
- 4.3 Operazioni di modifica
- 4.4 Operazioni di eliminazione
- 4.5 Operazioni di interrogazione
- 4.6 Operazioni statistiche / amministrative
- 4.7 Input, output e tabelle coinvolte per ogni operazione

5 Progettazione logica

- 5.1 Stima del volume dei dati
- 5.2 Descrizione delle operazioni principali e frequenza
- 5.3 Schemi di navigazione
- 5.4 Tabelle degli accessi
- 5.5 Analisi delle ridondanze
- 5.6 Raffinamento dello schema
 - 5.6.1 Eliminazione delle gerarchie
 - 5.6.2 Soluzione logica anticipata

La specializzazione totale ed esclusiva delle transazioni si traduce in una struttura logica con la relazione padre TRANSAZIONE e le relazioni figlie SPESA e ENTRATA. Questa scelta

mantiene separati gli attributi comuni da quelli specifici e rende esplicita la distinzione concettuale tra movimenti in uscita e movimenti in entrata.

5.6.3 Eliminazione degli attributi composti

5.6.4 Gestione degli attributi multivalore

5.6.5 Scelta delle chiavi primarie

5.6.6 Traduzione delle associazioni

5.7 Traduzione di entità e associazioni in relazioni

In questa fase si traducono le entità e le associazioni concettuali in relazioni logiche, specificando le chiavi primarie e le chiavi esterne necessarie per mantenere l'integrità referenziale.

5.8 Schema relazionale finale

Lo schema relazionale finale descrive le tabelle definitive, con i rispettivi attributi, tipi di dato e vincoli principali. Questa rappresentazione costituisce la base per la successiva fase di implementazione SQL.

5.9 Vincoli di integrità

In questa sezione si elencano i vincoli di integrità referenziali e di dominio che il database deve rispettare, come le chiavi primarie, le chiavi esterne, i vincoli UNIQUE e i controlli sui valori.

5.10 Controllo della normalizzazione

Qui si verifica che lo schema relazionale finale soddisfi i livelli di normalizzazione richiesti, identificando eventuali anomalie di inserimento, aggiornamento o cancellazione.

6 Implementazione SQL

6.1 Script di creazione del database

6.2 Creazione delle tabelle

6.3 Definizione di chiavi primarie e chiavi esterne

6.4 Vincoli NOT NULL, UNIQUE, CHECK

6.5 Trigger, procedure o viste, se usati

6.6 Popolamento iniziale del database

6.7 Query SQL delle operazioni principali

6.7.1 Problemi e criticità

Il progetto richiede di garantire l'integrità dei dati durante l'inserimento di una spesa o di un'entrata, e di gestire i budget in modo idempotente. Le principali criticità sono:

- Pericolo di orfani: l'inserimento nel padre (TRANSAZIONE) deve essere atomico rispetto all'inserimento nel figlio (SPESA o ENTRATA).
- Divisione per zero: nelle statistiche percentuali l'assenza di entrate potrebbe causare errori se non si utilizza una protezione adeguata.
- Gestione del budget: la coppia chiave (ID_Utente, ID_Categoria, Mese, Anno) deve essere unica per permettere work-flow di upsert senza duplicati.

6.7.2 Analisi

Il sistema utilizza una traduzione della gerarchia con una relazione per la super-entità TRANSAZIONE e una relazione per ciascuna sottoclasse, SPESA ed ENTRATA. Ogni inserimento in una sottoclasse richiede quindi la creazione preliminare della tupla nella relazione padre. Per i budget si introduce la logica di "upsert" per evitare duplicati e mantenere la coerenza del limite di spesa.

6.7.3 Soluzione SQL

```
1 -- Inserimento nel record padre
2 INSERT INTO TRANSAZIONE (Importo, Data, Descrizione, ID_Utente)
3 VALUES (?, ?, ?, ?);
4
5 -- Inserimento nel record figlio (JDBC recupera l'ID appena generato)
6 INSERT INTO SPESA (ID_Transazione, ID_Categoria)
7 VALUES (LAST_INSERT_ID(), ?);
```

Listing 1: Inserimento di una spesa con recupero ID

```
1 INSERT INTO BUDGET (Importo_Limite, ID_Utente, ID_Categoria, Mese, Anno)
2 VALUES (?, ?, ?, ?, ?)
3 ON DUPLICATE KEY UPDATE Importo_Limite = VALUES(Importo_Limite);
```

Listing 2: Upsert del budget per Categoria o Globale

```
1 SELECT
2     (COALESCE(SUM(CASE WHEN S.ID_Transazione IS NOT NULL THEN T.Importo
3         END), 0) /
4     NULLIF(COALESCE(SUM(CASE WHEN E.ID_Transazione IS NOT NULL THEN T.
5         Importo END), 0), 0)) * 100
6     AS Percentuale_Incidenza
7 FROM TRANSAZIONE T
8 LEFT JOIN SPESA S ON T.ID_Transazione = S.ID_Transazione
9 LEFT JOIN ENTRATA E ON T.ID_Transazione = E.ID_Transazione
10 WHERE T.ID_Utente = ? AND MONTH(T.Data) = ? AND YEAR(T.Data) = ?;
```

Listing 3: Rapporto Spese/Entrate Mensile

6.7.4 Query di supporto e operazioni amministrative

```
1 DELETE FROM TRANSAZIONE
2 WHERE ID_Transazione = ? AND ID_Utente = ?;
```

Listing 4: Eliminazione di transazione con ON DELETE CASCADE

```
1 UPDATE TRANSAZIONE
2 SET Importo = ?, Data = ?, Descrizione = ?
3 WHERE ID_Transazione = ? AND ID_Utente = ?;
```

Listing 5: Aggiornamento attributi base di una transazione

```

1 INSERT INTO APPARTENENZA_TAG (ID_Transazione, ID_Tag)
2 VALUES (?, ?);

```

Listing 6: Associazione Tag

```

1 INSERT INTO BUDGET (Importo_Limite, ID_Utente, ID_Categoria, Mese, Anno)
2 VALUES (?, ?, ?, ?, ?)
3 ON DUPLICATE KEY UPDATE Importo_Limite = VALUES(Importo_Limite);

```

Listing 7: Budget globale o specifico per categoria

```

1 SELECT T.ID_Transazione, T.Importo, T.Data,
2        COALESCE(C.Nome, F.Nome) AS Categoria_Fonte,
3        CASE WHEN S.ID_Transazione IS NOT NULL THEN 'Spesa' ELSE 'Entrata'
4        END AS Tipo
5 FROM TRANSAZIONE T
6 LEFT JOIN SPESA S ON T.ID_Transazione = S.ID_Transazione
7 LEFT JOIN CATEGORIA C ON S.ID_Categoria = C.ID_Categoria
8 LEFT JOIN ENTRATA E ON T.ID_Transazione = E.ID_Transazione
9 LEFT JOIN FONTE F ON E.ID_Fonte = F.ID_Fonte
10 WHERE T.ID_Utente = ? AND T.Data BETWEEN ? AND ?
11 ORDER BY T.Data DESC;

```

Listing 8: Riepilogo transazioni per periodo

```

1 SELECT C.Nome, SUM(T.Importo) AS Totale_Speso
2 FROM SPESA S
3 JOIN TRANSAZIONE T ON S.ID_Transazione = T.ID_Transazione
4 JOIN CATEGORIA C ON S.ID_Categoria = C.ID_Categoria
5 WHERE T.ID_Utente = ? AND T.Data BETWEEN ? AND ?
6 GROUP BY C.ID_Categoria, C.Nome
7 ORDER BY Totale_Speso DESC;

```

Listing 9: Categorie più costose per periodo

```

1 SELECT TA.Nome, COUNT(*) AS Numero_Utilizzi
2 FROM APPARTENENZA_TAG AT
3 JOIN TAG TA ON AT.ID_Tag = TA.ID_Tag
4 JOIN TRANSAZIONE TR ON AT.ID_Transazione = TR.ID_Transazione
5 WHERE TR.ID_Utente = ?
6 GROUP BY TA.ID_Tag, TA.Nome
7 ORDER BY Numero_Utilizzi DESC;

```

Listing 10: Tag più utilizzati

```

1 SELECT MONTH(T.Data) AS Mese, YEAR(T.Data) AS Anno, SUM(T.Importo) AS
   Totale
2 FROM TRANSAZIONE T
3 JOIN SPESA S ON T.ID_Transazione = S.ID_Transazione
4 JOIN BUDGET B ON T.ID_Utente = B.ID_Utente
5     AND B.Mese = MONTH(T.Data)
6     AND B.Anno = YEAR(T.Data)
7 WHERE T.ID_Utente = ? AND B.ID_Categoria IS NULL
8 GROUP BY YEAR(T.Data), MONTH(T.Data), B.Importo_Limite
9 HAVING SUM(T.Importo) > B.Importo_Limite;

```

Listing 11: Budget mensile complessivo superato

```

1 SELECT YEAR(T.Data) AS Anno, MONTH(T.Data) AS Mese, SUM(T.Importo) AS
   Totale
2 FROM TRANSAZIONE T
3 JOIN SPESA S ON T.ID_Transazione = S.ID_Transazione
4 WHERE T.ID_Utente = ?
5 GROUP BY YEAR(T.Data), MONTH(T.Data)
6 ORDER BY Anno DESC, Mese DESC;

```

Listing 12: Andamento delle spese mensili

```

1 SELECT C.Nome, SUM(T.Importo) AS Totale_Annuo
2 FROM SPESA S
3 JOIN TRANSAZIONE T ON S.ID_Transazione = T.ID_Transazione
4 JOIN CATEGORIA C ON S.ID_Categoria = C.ID_Categoria
5 WHERE T.ID_Utente = ? AND YEAR(T.Data) = ?
6 GROUP BY C.ID_Categoria, C.Nome
7 ORDER BY Totale_Annuo DESC
8 LIMIT 1;

```

Listing 13: Categoria con maggior spesa annuale

```

1 SELECT C.Nome, SUM(T.Importo) AS Totale
2 FROM SPESA S
3 JOIN TRANSAZIONE T ON S.ID_Transazione = T.ID_Transazione
4 JOIN CATEGORIA C ON S.ID_Categoria = C.ID_Categoria
5 WHERE T.ID_Utente = ? AND T.Data BETWEEN ? AND ?
6 GROUP BY C.ID_Categoria, C.Nome;

```

Listing 14: Distribuzione spese per due periodi

```

1 SELECT C.Nome, COUNT(*) AS Volte_Sforato
2 FROM (
3     SELECT S.ID_Categoria, MONTH(T.Data) AS Mese, YEAR(T.Data) AS Anno,
4     SUM(T.Importo) AS Somma
5     FROM SPESA S JOIN TRANSAZIONE T ON S.ID_Transazione = T.
6     ID_Transazione
7     WHERE T.ID_Utente = ?

```

```

6      GROUP BY S.ID_Categoria, MONTH(T.Data), YEAR(T.Data)
7  ) AS Spese_Mensili
8  JOIN BUDGET B ON Spese_Mensili.ID_Categoria = B.ID_Categoria
9      AND Spese_Mensili.Mese = B.Mese
10     AND Spese_Mensili.Anno = B.Anno
11     AND B.ID_Utente = ?
12  JOIN CATEGORIA C ON B.ID_Categoria = C.ID_Categoria
13  WHERE Spese_Mensili.Somma > B.Importo_Limite
14  GROUP BY C.ID_Categoria, C.Nome
15  ORDER BY Volte_Sforato DESC;

```

Listing 15: Categorie che superano il budget più spesso

6.7.5 Motivazioni

Atomicità applicativa: in Java, l'uso di `conn.setAutoCommit(false)` unito a queste query garantisce che la TRANSAZIONE esista solo se esiste anche la SPESA o l'ENTRATA.

Flessibilità budget: il vincolo `UNIQUE(ID_Utente, ID_Categoria, Mese, Anno)` permette alla query `ON DUPLICATE KEY UPDATE` di aggiornare o creare il limite in modo efficiente.

Robustezza matematica: `NULLIF` trasforma lo zero in `NULL`, evitando l'errore di divisione per zero nel database engine.

7 Progettazione dell'applicazione

7.1 Architettura generale dell'applicazione

7.2 Tecnologie usate

7.3 Struttura dei package Java

7.4 Collegamento al database tramite JDBC

7.5 Organizzazione delle classi principali

7.6 Gestione degli errori e validazione input

7.7 Interfaccia grafica Java Swing

7.8 Schermate principali

7.9 Flussi d'uso principali

Registrazione studente
Inserimento di una spesa
Inserimento di una entrata
Definizione di un budget
Gestione di una spesa ricorrente
Associazione di un documento digitale
Visualizzazione statistiche amministratore

8 Test e validazione

8.1 Dati di test utilizzati

8.2 Test delle query SQL

8.3 Test delle funzionalità applicative

8.4 Casi limite gestiti

8.5 Errori noti o limitazioni

9 Conclusioni

9.1 Risultato ottenuto

9.2 Scelte progettuali principali

9.3 Possibili sviluppi futuri

A Script SQL completo

B Query SQL complete

C Screenshot dell'applicazione

D Manuale utente sintetico

E Eventuale link al repository